## 1.1 Search Problems

- **Search processes:** {S: set of states} {s0: initial state} {A(s): set of actions, for each state s} {α: S→A policy, 若你在s状态,应该做动作a} {转移函数τ: S×A→S若你在状态s执行了动作a, 这个函数告诉你你会变成什么新状态} {c: S→R+, cost function} {G: ⊆S goal states}
- **Assumptions**: at each time-step t=0,...,T−1,agent will:

| observe | select | move |
|---|---|---|
| observe the state | select an action | act on the state |
| $S_t$ | $A_t = \alpha(S_t)$ | $(S_t, A_t)$ |

  - the environment evolves to a new state, $S_{t+1} = \tau(S_t, A_t)$
  - the agent incurs a cost $C_t = c(S_t, A_t, S_{t+1})$
- **Goal:**
  find an optimal policy, $\alpha^*$, to minimize the loss (total cost) $\sum_{t=0}^{T} C_t$ after $S_T \in \mathcal{G}$
- **Search Algorithm:** explore an open path / check for goal / expand or export chosen path
- **Code Example**:

```python
def search(graph: Graph, goals: Dict[Key, Vertex], start: Key) -> Optional[Path]:
    # helper
    def _search(graph: Graph goals: Dict[Key, Vertex], open: Queue[Path]) -> Optional[Path]:
        if open.size is 0:
            return None
        # explore
        path = open.pop()
        if goals.get(path.dst.key) is not None:
            return path
        for edge in graph.get_edges_from(path.dst.key):
            # expand
            new_path = path.expand(edge)
            # export
            open.push(new_path.len, new_path)
        return self._search(self, graph, goals, open)
    open = Queue()

    # create a dummy path
    dummy = Path()
    dummy.dst = start
    open.push(0, dummy)

    return self._search(goals, open)
```

~

- **We want algorithm to be**: halting (always returns a (possibly NULL) solution); complete (returns a non-NULL solution whenever a non-NULL solution exists); Sound (returns NULL whenever no solution exists); Optimal ; time-efficient ; space-efficient

| | breadth-first search<br>remove paths in increasing order of length | depth-first search<br>remove paths in decreasing order of depth | cheapest-first search<br>remove paths in increasing order of total cost |
|---|---|---|---|
| property | | | |
| halting | ✓ | ✗ | ✓ |
| complete | ✓ | ✗ | ✓ |
| optimal | ✗ | ✗ | ✓ |
| time | $O(b^d)$ | $O(b^d)$ | $O(b^{\lceil c^*/\varepsilon \rceil})$ |
| space | $O(b^{d+1})$ | $O(bd)$ | $O(b^{\lceil c^*/\varepsilon \rceil})$ |

- **Four terms for cycle-check:**
  -- 空间维度：你的"记性"有多好？
  - **Intra-path**:"我**这一趟**是怎么过来的"。防止在相同路径中重复访问顶点,AI 只记得"我这一趟是怎么过来的"。若下一步要去的点已在当前的路径里,就放弃。for **DFS**
  - **Inter-path**: 防止在**所有路径中重复**访问顶点。记得"全世界所有分身去过的点"。只要任何一个分身去过点 X，其他人就不准再去。for all (esp. **BFS, Dijkstra, A***
  -- 时间维度：你什么时候给点"打标签"？
  是在确定使用 Inter-path 检查后，决定在代码的哪一行检查
  - **Explore-time**: signed as visited only when a vertex is **Pop()** from Open Queue. (for **Dijkstra, A***
  - **Export-time**: signed as visited just when a vertex is **to be Push()** into Open Queue. (for **BFS**
- **Iterative Deepening & Iterative Inflating:**
  - **Iterative-deepening**: search to a fixed depth, increase the depth each iteration. for **DFS**
  - **Iterative-inflating**: search to a fixed cost, increase the cost each iteration. e.g. 进行搜索，时任何总花费超过 10 块钱的路径直接剪枝掉。(for **Dijkstra, A***

## 1.2 Heuristics

**Basic definition:**

- **h(s)**: 从状态 s 到目标状态的**估计**未来代价
- **h*(s)**: 从状态 s 到目标状态的**真实**最小代价
- **c̃(p)**: **估算总**代价 (即 f 值):

$$\boxed{\tilde{c}(p) = c(p) + h(p)}$$

- **c(p)**: 从初始状态 s0 到当前位置 dst(p) 的**实际已付**代价(即 g 值)
- **h(p)**: 从当前位置到目标的**估计**未来代价 {h(p)缩写自 h(dst(p))}

| ideal heuristic | over-estimate | under-estimate |
|---|---|---|
| $h^*(p)$ | $h(p) > h^*(p)$ | $h(p) < h^*(p)$ |
| actual cost from dst$(p)$ to $\mathcal{G}$ | estimate is greater than actual cost from dst$(p)$ to $\mathcal{G}$ | estimate is less than actual cost from dst$(p)$ to $\mathcal{G}$ |

It is always better for a heuristic to **under-estimate**

**Heuristic Properties:**

- **Admissible**: A heuristic, h(·), $\boxed{\textbf{h(s) ≤h*(s)}}$ for all s∈S
  为什么重要: 只要你不高估, A* 就一定能找到那条真正最省钱的路。如果你高估了某条好路，AI 可能会因为觉得它"贵"而错过它。
- **Consistent**:

$$\boxed{h(s) \le cost(s, s') + h(s')。}$$

$$\underbrace{h(s) - h(\mathrm{tr}(s,a))}_{\text{estimated cost of the transition}(s,a,\mathrm{tr}(s,a))} \le \underbrace{c(s,a,\mathrm{tr}(s,a))}_{\text{true cost of the transition, }(s,a,\mathrm{tr}(s,a))}$$

A→B: for all $s \in \mathcal{S}$, and $a \in \mathcal{A}(s)$, and $h(s) = 0$ for $s \in \mathcal{G}$.

A 点的估计 <= 走这一步的花费 + B 点的估计

为什么重要: 若h是一致的，A* 算法在搜索时，路径的f值 (已花+预估) 会越走越高 (或者不变)，永远不会往下跌。这样 AI 就不用走回头路去重新修正之前的错误估计。

**Domination:**

- h1 **Strongly dominates** h2 if for all s∈S\G: **h1(s) >h2(s)**
- h1 **Weakly dominates** h2 if for all s∈S: **h1(s) ≥h2(s)** and for some s∈S: h1(s) >h2(s)

**Combining:**

If $\{h_k(\cdot)\}_k$ are admissible (resp. consistent), then $\max_k \{h_k\}(\cdot)$ is also admissible (resp. consistent) and weakly dominates.

**Nearest-first search v.s. A*-search:** halting, complete, opt

- NFS (c̃ = h) v.s A* Search (c̃ = c + h)

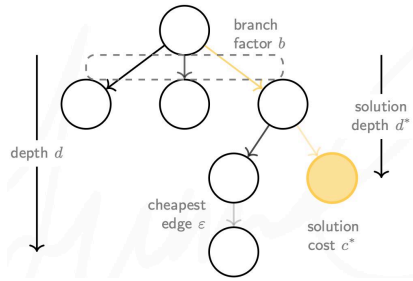
**Problem Relaxation**

- **逻辑**:
  1. 定义原始问题及其完美代价 $h^*_{ori}$ 。
  2. 去掉或放宽一些约束条件，得到"松弛问题" 。
  3. 松弛问题的最优代价 $c_{rel}$ 一定满足 $c_{rel}(s) \le h^*_{ori}(s)$ 。
  4. **结论**: 松弛问题的解 $c_{rel}$ 可以直接作为原问题的**可采纳启发式函数** 。
- **实例**: 推箱子中忽略墙壁，计算曼哈顿距离 。 +1

**Eg. If a heuristic, h(·), is consistent, then it is also admissible.**

**Proof: Consistency ⟹ Admissibility**

1. **Base Case**: $h(G) = 0 \le h^*(G) = 0$.
2. **IH**: Assume $h(s_k) \le h^*(s_k)$ for $k$ steps to goal.
3. **Step**: For $s_{k+1}$, by consistency: $h(s_{k+1}) \le c(a) + h(s_k)$.
4. Substitute IH: $h(s_{k+1}) \le c(a) + h^*(s_k)$.
5. By definition: $c(a) + h^*(s_k) = h^*(s_{k+1})$.
6. Result: $h(s_{k+1}) \le h^*(s_{k+1})$. QED.



# 2. Constraint Satisfaction Problems

- **Variables:** V= {V1,V2,...,Vn}
- **Domain:** Di, for each variable Vi
- **Constraints:** C= {C1,C2,...,Cm}

we can model in two ways:

domains: $\bullet \notin \mathrm{dom}(V_2)$ constraints $V_2 \neq \bullet$

$\bullet \notin \mathrm{dom}(V_4)$ $V_4 \neq \bullet$

Binary constraints: 涉及 2 个变量（如 $V_1 < V_2$）

**constraint satisfaction algorithms:**

1. **检查解 (Check Solution)**：是否所有变量都已赋值？是则返回 。
2. **选择变量 (Select Variable)**：挑一个还没赋值的变量 $V_i$ 。
3. **尝试赋值 (Try Value)**：从值域 $D_i$ 中选一个值 $v$ 赋给 $V_i$ 。
4. **一致性检查 (Check Consistency)**：这个赋值是否违反了现有的约束？
   - 如果违反了 $\Longrightarrow$ **回溯 (Backtrack)**，尝试下一个值 。 +1
5. **传播 (Propagate)**：根据当前的赋值，更新其他未赋值变量的值域（这是提速的关键）

**Three ways for propagate:**

| | | | |
|---|---|---|---|
| Backtracking (BT) | 赋值后 | 只检查当前值与**已赋值**变量的冲突。 | **开销最低**，但发现失败最晚，易产生指数级回溯。 |
| Forward-Checking (FC) | 赋值后 | 检查当前变量对**未赋值邻居**的影响。 | **中等开销**。通过划掉邻居的不可能选项，若邻居值域为空则立即回溯。 |
| Arc-Consistency (AC) | 赋值后或预处理 | 检查**所有弧** $(X, Y)$。若 $X$ 的值在 $Y$ 中无对应匹配，则删掉该值。 | **开销最高**。产生连锁反应（多米诺效应），能最早发现深层矛盾。 |

**Three ways to select the order:**

- **minimum remaining values MRV:** 选剩余合法值最少的变量。
“Fail-first”。既然它最容易失败，就先试它；如果它注定失败，能帮你砍掉大片无效分支。是用来选 **下一个变量 (Variable)** 的
- **least constraining value LCV:** 选一个给未来变量留路最多的值。
“Succeed-first”。目标是找一个解，所以先走最平坦的路。
是用来为选定的变量选 **下一个值 (Value)** 的
- **most restrictive constraint:** 优先检查满足条件最少的“严苛”约束。及早剔除不合格赋值。(如果有条规矩特别奇葩，全天下只有 1% 的组合能过，那你就先拿这条规矩去卡你的赋值。)
- **第一步（MRV 登场）**：“我手头有 100 个还没填色的国家，我该先翻哪一个人的牌子？” → **选变量**。
- **第二步（LCV 登场）**：“好，我已经决定先填加拿大了。加拿大有红、绿、蓝三种可选，我该按什么顺序试这三个色？” → **选值的顺序**。

所以，它们是**串行**的关系：MRV 帮你锁定“谁”，LCV 帮你锁定“怎么试”。

**arc-consistent (as property)**

我们说弧 $\langle X, c(X,Y)\rangle$ **是弧一致的**（注意方向是从 $X$ 到 $Y$），当且仅当：

对于变量 $X$ 的值域中的**每一个**值 $x$，在变量 $Y$ 的值域中都**至少能找到一个**值 $y$，使得 $(x, y)$ 满足约束。

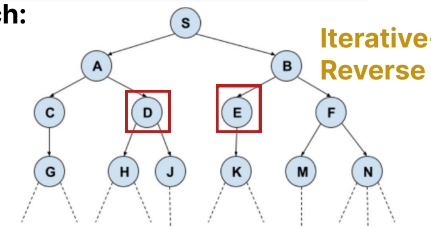
We want to schedule five activities in four time slots. Let the variables A, B, C, D, and E represent the activities. Each variable's domain is {1, 2, 3, 4}. The constraints are below.

$A \leq B$ $\quad A > D$ $\quad C \neq D$ $\quad C > E$

$A \neq C$ $\quad B \neq C$ $\quad C \neq D + 1$ $\quad D > E$

| Step | A | B | C | D | E | What Next? |
|---|---|---|---|---|---|---|
| 1 | 2, 3, 4 | 2, 3, 4 | C = 1 | 2, 3, 4 | empty | Backtrack |
| 2 | 1, 2, 3 | 1, 2, 3 | C = 4 | 1, 2 | 1, 2, 3 | Continue |
| 3 | 2, 3 | 1, 2, 3 | | D = 1 | empty | Backtrack |
| 4 | 3 | 1, 2, 3 | | D = 2 | 1 | Continue |
| Apply the MRV heuristic. Both A and E have 1 value left. Choose A by the tie-breaking rule. | | | | | | |
| 5 | A = 3 | 3 | | | 1 | Continue |
| Apply the MRV heuristic. Both B and E have 1 value left. Choose B by the tie-breaking rule. | | | | | | |
| 6 | | B = 3 | | | 1 | Continue |
| 7 | | | | | E = 1 | Solution |

**Ex. Search:**



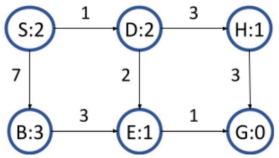

(search graph continues below this level)

**Iterative-Deepening**
**Reverse alphabetical**

- **S** is the initial state.
- **D** and **E** are the goal states.

| Step | Depth limit | Node to expand | Nodes to add | Frontier |
|---|---|---|---|---|
| 1 | 0 | None | S | S |
| 2 | 0 | S | None | None |
| 3 | 1 | None | S | S |
| 4 | 1 | S | B A | B A |
| 5 | 1 | A | None | B |
| 6 | 1 | B | None | None |
| 7 | 2 | None | S | S |
| 8 | 2 | S | B A | B A |
| 9 | 2 | A | D C | B D C |
| 10 | 2 | C | None | B D |
| 11 | 2 | D | | |
| 12 | | | | |

**Ex. Search:**




| Step | State | g(State) | Frontier (State + Its f value) |
|---|---|---|---|
| 1 | S(2) | 0 | SD(3), SB(10) |
| 2 | SD(3) | 1 | SDE(4), SDH(5), SB(10) |
| 3 | SDE(4) | 3 | SDEG(4), SDH(5), SB(10) |
| 4 | SDEG(4) | 4 | |
| 5 | | | |
| 6 | | | |

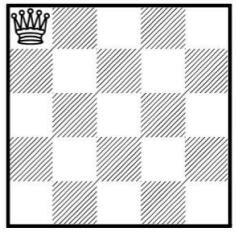
**Ex. Constraints Modeling in map:**

Figure 1: A map-coloring problem.

- Variables: $WA, NT, SA, Q, NSW, V, T$
- Domains: $D = \{red, green, blue\}$
- Constraints: $WA \neq NT$, $WA \neq SA$, $NT \neq SA$, $NT \neq Q$, $SA \neq Q$, $NSW \neq V$ $\quad SA \neq NSW$, $SA \neq V$, $Q \neq NSW$,

**3. Arc Consistency**

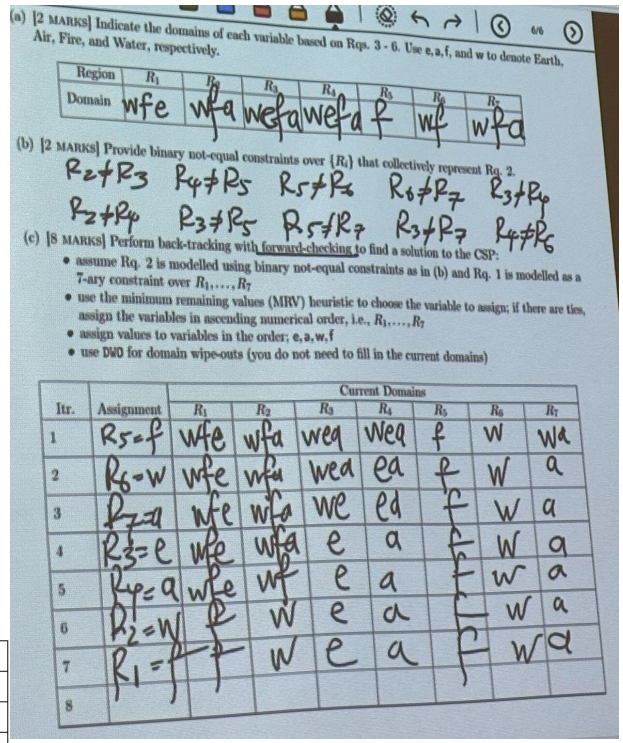
Consider the 5-Queens problem. Recall that the rows are numbered 1 to 5 from top to bottom. $Q_k$ refers to the queen in row k. Assume that all variables started with the domain {1,2,3,4,5}.



i. each person receives no more than one item

**Solution:**

$$\forall i \in N : \sum_{g_j \in G} x_{i,j} \leq 1$$



(a) [2 MARKS] Indicate the domains of each variable based on Rqs. 3 - 6. Use e, a, f, and w to denote Earth, Air, Fire, and Water, respectively.

| Region | $R_1$ | $R_2$ | $R_3$ | $R_4$ | $R_5$ | $R_6$ | $R_7$ |
|---|---|---|---|---|---|---|---|
| Domain | wfe | wfa | wefa | wefa | f | wf | wfa |

(b) [2 MARKS] Provide binary not-equal constraints over $\{R_i\}$ that collectively represent Rq. 2.

$R_2 \neq R_3$ $\quad R_4 \neq R_5$ $\quad R_5 \neq R_6$ $\quad R_6 \neq R_7$ $\quad R_3 \neq R_4$

$R_2 \neq R_4$ $\quad R_3 \neq R_5$ $\quad R_5 \neq R_7$ $\quad R_3 \neq R_7$ $\quad R_4 \neq R_6$

(c) [8 MARKS] Perform back-tracking with forward-checking to find a solution to the CSP:

- assume Rq. 2 is modelled using binary not-equal constraints as in (b) and Rq. 1 is modelled as a 7-ary constraint over $R_1, \ldots, R_7$
- use the minimum remaining values (MRV) heuristic to choose the variable to assign; if there are ties, assign the variables in ascending numerical order, i.e., $R_1, \ldots, R_7$
- assign values to variables in the order; e, a, w, f
- use DWO for domain wipe-outs (you do not need to fill in the current domains)

| Itr. | Assignment | Current Domains $R_1$ | $R_2$ | $R_3$ | $R_4$ | $R_5$ | $R_6$ | $R_7$ |
|---|---|---|---|---|---|---|---|---|
| 1 | $R_5 = f$ | wfe | wfa | wea | wea | f | w | wa |
| 2 | $R_6 = w$ | wfe | wfa | wea | ea | f | w | a |
| 3 | $R_7 = a$ | wfe | wfa | we | ea | f | w | a |
| 4 | $R_3 = e$ | wfe | wfa | e | a | f | w | a |
| 5 | $R_4 = a$ | wfe | wf | e | a | f | w | a |
| 6 | $R_2 = w$ | f | w | e | a | f | w | a |
| 7 | $R_1 = f$ | f | w | e | a | f | w | a |
| 8 | | | | | | | | |